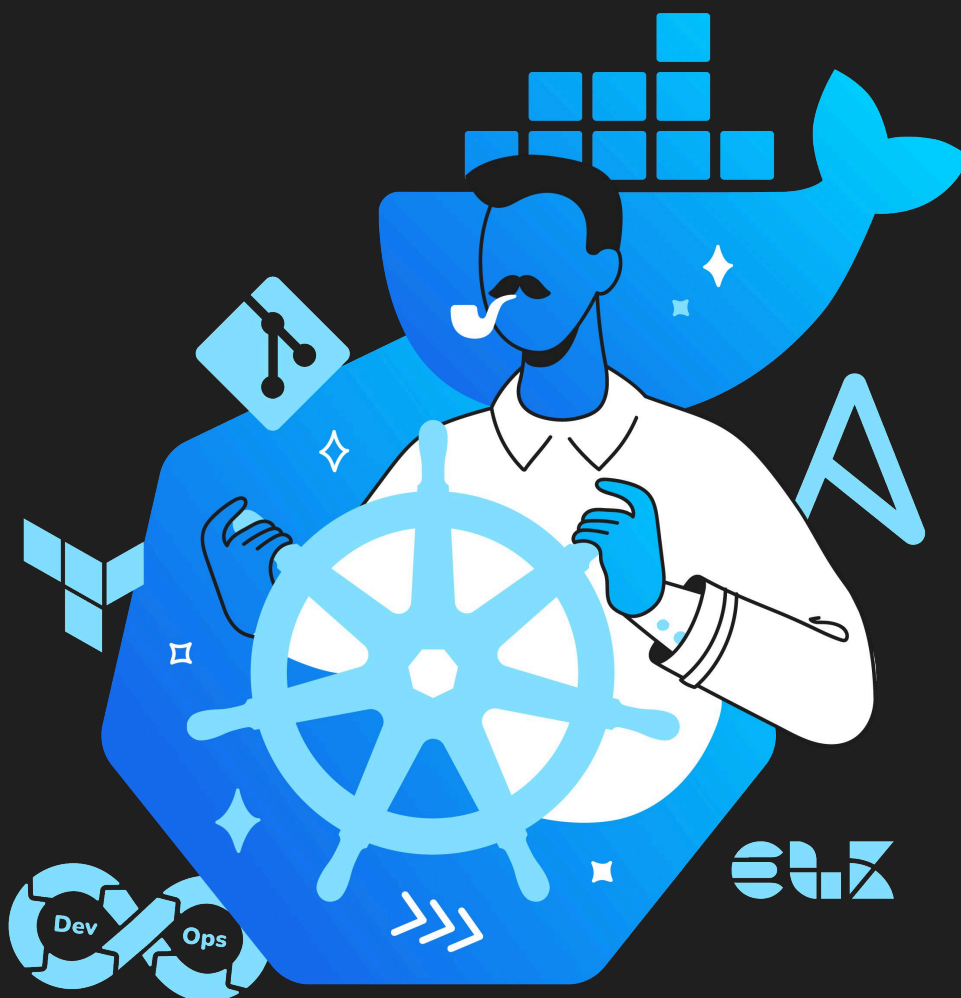


Методический материал

КОННЕКТ BRIDGE-СЕТЕЙ И ИСПОЛЬЗОВАНИЮ CAP_NET_ADMIN

Урок 7.





Создадим Docker network с использованием Bridge network driver:

```
$ docker network create --driver bridge merion-network-1
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker network create --driver bridge merion-network-1
3537b1e4090a71ea65122980fe924d5cc0fc72ee9635169209b5129534247526
```

Увидеть созданную сеть можно с помощью команды:

```
$ docker network ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
fab1e211f705        bridge             bridge              local
bfde5bf92be3        host               host                local
3537b1e4090a        merion-network-1   bridge              local
bf4cd927de56        none               null                local
```

Запустим контейнер с именем "Container-1" в созданной сети и сразу перейдем в псевдо-терминал:

```
$ docker run -it --rm --network merion-network-1 --name
container-1 alpine:3.15
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker run -it --rm --network merion-network-1
--name container-1 alpine:3.15
/ #
```



Выполним команду по добавлению пакета Iptables:

```
$ apk add iptables
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker run -it --rm --network merion-network-1
--name container-1 alpine:3.15
/ # apk add iptables
fetch https://dl-cdn.alpinelinux.org/alpine/v3.15/main/x86_64/APKINDEX.tar.gz
fetch https://dl-cdn.alpinelinux.org/alpine/v3.15/community/x86_64/APKINDEX.tar.gz
(1/3) Installing libmnl (1.0.4-r2)
(2/3) Installing libnftnl (1.2.1-r0)
(3/3) Installing iptables (1.8.7-r1)
Executing busybox-1.34.1-r7.trigger
OK: 8 MiB in 17 packages
/ # █
```

Выполним команду по просмотру списка сетевых правил:

```
$ iptables -L
```

```
/ # iptables -L
iptables v1.8.7 (legacy): can't initialize iptables table `filter': Permission
denied (you must be root)
Perhaps iptables or your kernel needs to be upgraded.
/ # █
```

Получаем ошибку "Permission denied".

Пока не будем останавливать "Container-1", он пригодится нам в дальнейшем.



Запустим контейнер с именем "Container-netadm" с Capability CAP_NET_ADMIN:

```
$ docker run -it --rm --network merion-network-1 --name  
container-netadm --cap-add CAP_NET_ADMIN alpine:3.15
```

```
~: docker  
filipp@user-IdeaPad-3-15IAU7:~$ docker run -it --rm --network merion-network-1 --name c  
ontainer-netadm --cap-add CAP_NET_ADMIN alpine:3.15  
/ #
```

Установим пакет Iptables и выполним команду просмотра сетевых правил:

```
$ apk add iptables  
$ iptables -L
```

```
/ # apk add iptables  
fetch https://dl-cdn.alpinelinux.org/alpine/v3.15/main/x86_64/APKINDEX.tar.gz  
fetch https://dl-cdn.alpinelinux.org/alpine/v3.15/community/x86_64/APKINDEX.tar.gz  
(1/3) Installing libmnl (1.0.4-r2)  
(2/3) Installing libnftnl (1.2.1-r0)  
(3/3) Installing iptables (1.8.7-r1)  
Executing busybox-1.34.1-r7.trigger  
OK: 8 MiB in 17 packages  
/ # iptables -L  
Chain INPUT (policy ACCEPT)  
target      prot opt source          destination  
  
Chain FORWARD (policy ACCEPT)  
target      prot opt source          destination  
  
Chain OUTPUT (policy ACCEPT)  
target      prot opt source          destination  
/ #
```



Остановим контейнер сочетанием клавиш CTRL+D.

Создадим еще одну Docker network с использованием Bridge network driver:

```
$ docker network create --driver bridge merion-network-2
```

```
~ : bash
filipp@user-IdeaPad-3-15IAU7:~$ docker network create --driver bridge merion-network-2
924b98d634108050354bd7522bff5f05773978c6757e186615a0c182c28bbd18
```

Увидеть созданную сеть можно с помощью команды:

```
$ docker network ls
```

```
~ : bash
filipp@user-IdeaPad-3-15IAU7:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
fab1e211f705        bridge              bridge              local
bfde5bf92be3        host                host                local
3537b1e4090a        merion-network-1    bridge              local
924b98d63410        merion-network-2    bridge              local
bf4cd927de56        none                null                local
```



Запустим контейнер с именем "Container-2" в созданной сети:

```
$ docker run -it --rm --network merion-network-2 --name container-2 alpine:3.15
```

```
~: docker
filipp@user-IdeaPad-3-15IAU7:~$ docker run -it --rm --network merion-network-2 --name container-2 alpine:3.15
/ #
```

На хосте выполним команды для инспектирования двух созданных нами Docker networks:

```
$ docker network inspect merion-network-1
```

```
"Containers": {
  "34528eda6b1de0a9597ca45bc10e39cad6e42c1223e5ad512ebed510fe12dca9": {
    "Name": "container-1",
    "EndpointID": "05649fec85d274b3865027e37e856009f3a69517d9d1cd907872e1c4d64085dd",
    "MacAddress": "ba:84:43:60:20:4d",
    "IPv4Address": "172.19.0.2/16",
    "IPv6Address": ""
  }
}
```



Попробуем из псевдо-терминала Container-1 в сети Merion-network-1 пропинговать Container-2 в сети Merion-network-2 и наоборот (используйте полученные в предыдущем шаге IPv4Address контейнеров Container-1 и Container-2:

- (Container-1):

```
$ ping 172.18.0.2
```

- (Container-2):

```
$ ping 172.19.0.2
```

The first terminal window shows a successful ping from Container-1 to Container-2: `/ # ping 172.18.0.2` followed by `PING 172.18.0.2 (172.18.0.2): 56 data bytes` and a cursor. The second terminal window shows a successful ping from Container-2 to Container-1: `/ # ping 172.19.0.2` followed by `PING 172.19.0.2 (172.19.0.2): 56 data bytes` and a cursor.

Получаем потери пакетов для обоих пингов:

The first terminal window shows a failed ping from Container-1 to Container-2: `container-1 alpine:3.15`, `/ # ping 172.18.0.2`, `PING 172.18.0.2 (172.18.0.2): 56 data bytes`, `^C`, `--- 172.18.0.2 ping statistics ---`, `106 packets transmitted, 0 packets received, 100% packet loss`, `/ #`. The second terminal window shows a failed ping from Container-2 to Container-1: `/ # ping 172.19.0.2`, `PING 172.19.0.2 (172.19.0.2): 56 data bytes`, `^C`, `--- 172.19.0.2 ping statistics ---`, `80 packets transmitted, 0 packets received, 100% packet loss`, `/ #`.



Оба контейнера не останавливаем, далее выполним коннект контейнеров к нужным Bridge-сетям.

На хосте выполним коннект Container-1 с сетью Merion-network-2 и коннект Container-2 с сетью Merion-network-1:

```
$ docker network connect merion-network-2 container-1  
$ docker network connect merion-network-1 container-2
```

```
~ : bash  
filipp@user-IdeaPad-3-15IAU7:~$ docker network connect merion-network-2 container-1  
filipp@user-IdeaPad-3-15IAU7:~$ docker network connect merion-network-1 container-2  
filipp@user-IdeaPad-3-15IAU7:~$
```



Повторим процедуру пинга между контейнерами:

- (Container-1):

```
$ ping 172.18.0.2
```

- (Container-2):

```
$ ping 172.19.0.2
```

```
~ : docker
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.173 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.171 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.252 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.196 ms
64 bytes from 172.18.0.2: seq=4 ttl=64 time=0.161 ms
64 bytes from 172.18.0.2: seq=5 ttl=64 time=0.239 ms
64 bytes from 172.18.0.2: seq=6 ttl=64 time=0.184 ms
64 bytes from 172.18.0.2: seq=7 ttl=64 time=0.334 ms
█

~ : docker
/ # ping 172.19.0.2
PING 172.19.0.2 (172.19.0.2): 56 data bytes
64 bytes from 172.19.0.2: seq=0 ttl=64 time=0.172 ms
64 bytes from 172.19.0.2: seq=1 ttl=64 time=0.150 ms
64 bytes from 172.19.0.2: seq=2 ttl=64 time=0.196 ms
64 bytes from 172.19.0.2: seq=3 ttl=64 time=0.236 ms
64 bytes from 172.19.0.2: seq=4 ttl=64 time=0.202 ms
64 bytes from 172.19.0.2: seq=5 ttl=64 time=0.276 ms
█
```



Пинги проходят, сетевая доступность для контейнеров с нужными Bridge-сетями установлена:

```
~ : docker
64 bytes from 172.18.0.2: seq=24 ttl=64 time=0.279 ms
64 bytes from 172.18.0.2: seq=25 ttl=64 time=0.242 ms
64 bytes from 172.18.0.2: seq=26 ttl=64 time=0.085 ms
64 bytes from 172.18.0.2: seq=27 ttl=64 time=0.055 ms
^C
--- 172.18.0.2 ping statistics ---
28 packets transmitted, 28 packets received, 0% packet loss
round-trip min/avg/max = 0.055/0.233/0.450 ms
/ # █

~ : docker
64 bytes from 172.19.0.2: seq=20 ttl=64 time=0.203 ms
64 bytes from 172.19.0.2: seq=21 ttl=64 time=0.287 ms
64 bytes from 172.19.0.2: seq=22 ttl=64 time=0.269 ms
64 bytes from 172.19.0.2: seq=23 ttl=64 time=0.202 ms
64 bytes from 172.19.0.2: seq=24 ttl=64 time=0.163 ms
^C
--- 172.19.0.2 ping statistics ---
25 packets transmitted, 25 packets received, 0% packet loss
round-trip min/avg/max = 0.059/0.200/0.287 ms
/ # █
```

В качестве завершающего действия можно остановить контейнеры сочетанием клавиш "CTRL+D" и выполнить удаление более неиспользуемых сетей:

```
$ docker network prune
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker network prune
WARNING! This will remove all custom networks not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Networks:
merion-network-2
merion-network-1
```



Все неиспользуемые сети были успешно удалены:

```
$ docker network ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
fab1e211f705        bridge             bridge              local
bfde5bf92be3        host               host                local
bf4cd927de56        none              null                local
```